

Churn Prediction with Tree-Based Models and Fairness Assessment

Killian Le Milbeau

15 décembre 2025

Table des matières

1	Introduction and Data overview	1
2	Models	2
2.1	Baseline	2
2.2	Feature engineering	2
2.3	Feature importance	3
2.4	Hyperparameter tuning and Nested CV	4
3	Fairness Analysis (Gender)	5
3.1	Definitions	5
3.2	Results and gaps	5
4	Conclusion and remark	6

1 Introduction and Data overview

This first Machine Learning project addresses a classical binary classification task : "predicting" whether a customer will churn (1) using demographic and account features. The Dataset contains 8,000 rows and 11 variables, that contain both numerical variables (CreditScore, Age, Balance, Salary...) and categorical variables (Geography, Gender).

The target is imbalanced : 6,391 non-churners vs. 1,609 churners (i.e. $\sim 20\%$ churn). Because of this imbalance, we will focus on AUC-ROC (with Accuracy as a secondary metric) and use Stratified Cross-Validation for robust comparisons.

From the descriptive statistics at the beginning of the notebook, and as an example of interpretable signal of raw data, churners are older on average than non-churners. This observation will motivate non-linear models and a little bit of feature engineering/transformation.

Since fairness is evaluated with respect to Gender, we also report gender composition in churn vs. non-churn groups.

2 Models

We begin with 5 models that covers a large parts of the possible Machine Learning algorithms : Logistic Regression, Random Forest, Support Vector Machine (SVM), Light Gradient Boosting Model (LightGBM) and Xtreme Gradient Boosting (XGBoost). These models are not choosed randomly. Logistic Regression is the first model we think of, simple and interpretable. Random Forest and XGBoost are often displaying very good performances, in Kaggle competitions they often outperform even NNs.

2.1 Baseline

We first implemented these 5 models on "raw" data to have a first idea about their respective performances. Then, for more robustness and to preserve class proportions, we compared these 5 standard models using 5-fold StratifiedKFold. This baseline comparison showed that tree ensembles dominate and that XGBoost is slightly ahead in term fo AUC-ROC. Therefore, we focused on Random Forest, LightGBM and XGBoost.

```
=====
Validating: Logistic Regression
=====
Mean Accuracy: 0.8114 (+/- 0.0061)
Mean AUC-ROC: 0.7550 (+/- 0.0147)

=====
Validating: Random Forest
=====
Mean Accuracy: 0.8620 (+/- 0.0056)
Mean AUC-ROC: 0.8487 (+/- 0.0106)

=====
Validating: SVM
=====
Mean Accuracy: 0.8575 (+/- 0.0070)
Mean AUC-ROC: 0.8207 (+/- 0.0131)

=====
Validating: LightGBM
=====
Mean Accuracy: 0.8601 (+/- 0.0082)
Mean AUC-ROC: 0.8555 (+/- 0.0081)

=====
Validating: XGBoost
=====
Mean Accuracy: 0.8619 (+/- 0.0059)
Mean AUC-ROC: 0.8568 (+/- 0.0082)
```

2.2 Feature engineering

We then construct engineered variables capturing non-linearities and interactions (age groups/senior, balance ratios, engagement features, and interaction terms).

We re-evaluate RF/LGBM/XGB on the engineered feature set using the same StratifiedKFold protocol. This establishes whether feature engineering improves performance before any tuning.

After adding engineered features performance does not improve automatically. For instance, XGBoost drops in AUC-ROC and Accuracy when moving from baseline features to engineered features with default-ish hyperparameters. This indicates that the new feature space increases complexity and must be paired with appropriate regularization / capacity control.

```

=====
Validating: Random Forest (FE)
=====
Mean Accuracy: 0.8626 (+/- 0.0061)
Mean AUC-ROC: 0.8438 (+/- 0.0133)

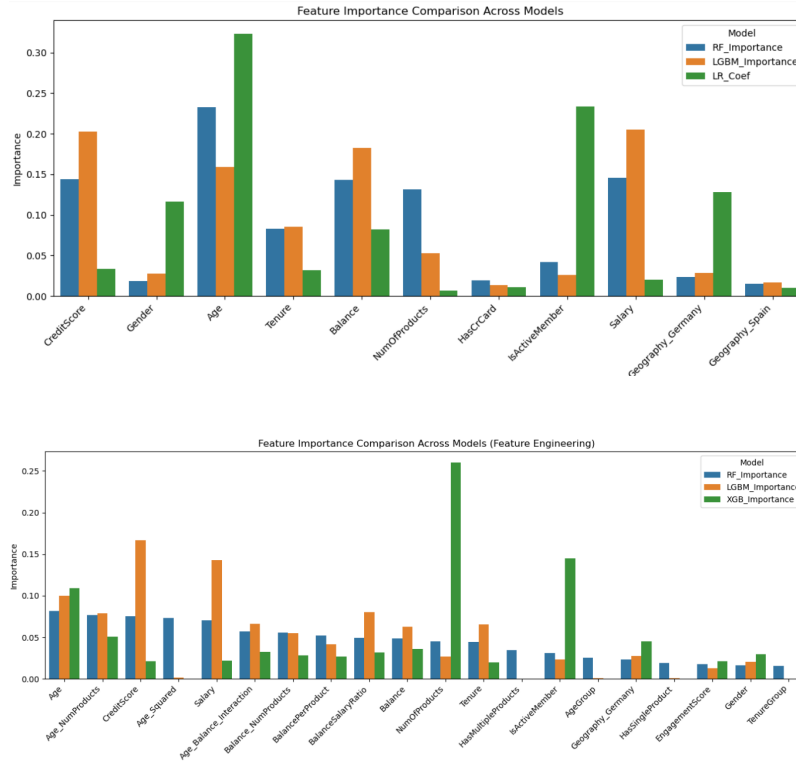
=====
Validating: LightGBM (FE)
=====
Mean Accuracy: 0.8605 (+/- 0.0059)
Mean AUC-ROC: 0.8519 (+/- 0.0083)

=====
Validating: XGBoost (FE)
=====
Mean Accuracy: 0.8589 (+/- 0.0077)
Mean AUC-ROC: 0.8531 (+/- 0.0083)

```

2.3 Feature importance

To provide a global understanding of model behavior, we report and visualize feature importance comparisons across models on the engineered feature set. This is not causal, but it supports discussion about which variables drive predictions most strongly.



2.4 Hyperparameter tuning and Nested CV

To reduce the risk of overly optimistic results from tuning, and to be safe against overfitting, we use a nested stratified cross-validation protocol : an inner CV loop for hyperparameter search, an outer CV loop for performance estimation.

```
=====
NESTED CV (outer=5 folds, inner=3 folds) - Random Forest (FE)
=====
Nested Mean Accuracy: 0.8643 (+/- 0.0027)
Nested Mean AUC-ROC: 0.8572 (+/- 0.0111)
Best params (outer folds) - first 2 folds:
  Fold 1: {'classifier__n_estimators': 400, 'classifier__min_samples_leaf': 4, 'classifier__max_features': 'sqrt', 'classifier__max_depth': 10}
  Fold 2: {'classifier__n_estimators': 200, 'classifier__min_samples_leaf': 2, 'classifier__max_features': 'log2', 'classifier__max_depth': 10}

Fitting FINAL tuned model on full FE data...
Final best AUC (inner CV): 0.8582946713851087
Final best params: {'classifier__n_estimators': 400, 'classifier__min_samples_leaf': 4, 'classifier__max_features': 'sqrt', 'classifier__max_depth': 10}

=====
NESTED CV (outer=5 folds, inner=3 folds) - LightGBM (FE)
=====
Nested Mean Accuracy: 0.8618 (+/- 0.0083)
Nested Mean AUC-ROC: 0.8574 (+/- 0.0094)
Best params (outer folds) - first 2 folds:
  Fold 1: {'classifier__subsample': 0.9, 'classifier__num_leaves': 31, 'classifier__n_estimators': 200, 'classifier__max_depth': 5, 'classifier__learning_rate': 0.01, 'classifier__colsample_bytree': 0.9}
  Fold 2: {'classifier__subsample': 0.9, 'classifier__num_leaves': 31, 'classifier__n_estimators': 200, 'classifier__max_depth': 5, 'classifier__learning_rate': 0.01, 'classifier__colsample_bytree': 0.9}

Fitting FINAL tuned model on full FE data...
Final best AUC (inner CV): 0.8618312764798265
Final best params: {'classifier__subsample': 0.9, 'classifier__num_leaves': 31, 'classifier__n_estimators': 300, 'classifier__max_depth': 20, 'classifier__learning_rate': 0.01, 'classifier__colsample_bytree': 0.9}

=====
NESTED CV (outer=5 folds, inner=3 folds) - XGBoost (FE)
=====
Nested Mean Accuracy: 0.8621 (+/- 0.0068)
Nested Mean AUC-ROC: 0.8640 (+/- 0.0102)
Best params (outer folds) - first 2 folds:
  Fold 1: {'classifier__subsample': 0.9, 'classifier__reg_lambda': 5.0, 'classifier__n_estimators': 500, 'classifier__min_child_weight': 1, 'classifier__max_depth': 5, 'classifier__learning_rate': 0.01, 'classifier__colsample_bytree': 0.7}
  Fold 2: {'classifier__subsample': 0.9, 'classifier__reg_lambda': 5.0, 'classifier__n_estimators': 500, 'classifier__min_child_weight': 1, 'classifier__max_depth': 5, 'classifier__learning_rate': 0.01, 'classifier__colsample_bytree': 0.7}

Fitting FINAL tuned model on full FE data...
Final best AUC (inner CV): 0.8622673992354226
Final best params: {'classifier__subsample': 0.9, 'classifier__reg_lambda': 5.0, 'classifier__n_estimators': 500, 'classifier__min_child_weight': 1, 'classifier__max_depth': 5, 'classifier__learning_rate': 0.01, 'classifier__colsample_bytree': 0.7}
```

Finally, we report the nested CV ranking (by AUC) of the three models and select the best one. Crucially, nested CV shows that tuning + FE produces a net gain over the baseline XGBoost while accuracy remains essentially unchanged. This confirms that feature engineering is beneficial only once the model is properly tuned. We therefore select XGBoost (feature engineering + tuned hyperparameters) as the final model, prioritizing AUC-ROC as the main criterion given the imbalance and the churn-detection objective.

	Model	Nested_Acc_mean	Nested_Acc_std	Nested_AUC_mean
2	XGBoost (FE)	0.862125	0.006796	0.864030
1	LightGBM (FE)	0.861750	0.008286	0.857415
0	Random Forest (FE)	0.864250	0.002721	0.857180

	Nested_AUC_std
2	0.010163
1	0.009428
0	0.011131

3 Fairness Analysis (Gender)

Fairness is assessed with respect to Gender, using out-of-fold (OOF) predictions from 5-fold StratifiedKFold to avoid in-sample bias when comparing groups.

3.1 Definitions

Let A be Gender, Y be the true churn label and $\hat{Y}$ the predicted label.

- **Independence** : Compare $\mathbb{P}(\hat{Y} = 1|A)$ across groups (demographic parity) ;
- **Separation** : Compare TPR $\mathbb{P}(\hat{Y} = 1|Y = 1, A)$ and FPR $\mathbb{P}(\hat{Y} = 1|Y = 0, A)$ across groups (equalized odds) ;
- **Sufficiency** : Compare PPV $\mathbb{P}(Y = 1|\hat{Y} = 1, A)$ (and NPV) across groups, and check the score calibration by group.

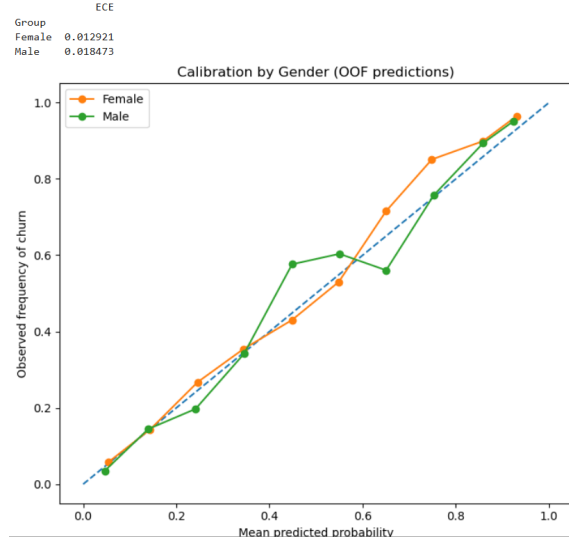
3.2 Results and gaps

We report : per-group base rates, per-group Independence/Separation/Sufficiency metrics, aggregate gaps (max-min) and Disparate Impact, calibration by group (ECE + calibration plot).

	n	BaseRate_P($\hat{Y}=1 A$) \	
Female	3627.0	0.247312	
Male	4373.0	0.162817	
GAP(max-min)	746.0	0.084495	
DisparateImpact(min/max PPR)	NaN	NaN	

	Independence_P($\hat{Y}=1 A$)	Separation_TPR \	
Female	0.147781	0.471572	
Male	0.092156	0.421348	
GAP(max-min)	0.055624	0.050224	
DisparateImpact(min/max PPR)	0.623603	NaN	

	Separation_FPR	Sufficiency_PPV	Sufficiency_NPV
Female	0.041392	0.789179	0.846652
Male	0.028134	0.744417	0.896222
GAP(max-min)	0.013258	0.044762	0.049570
DisparateImpact(min/max PPR)	NaN	NaN	NaN



To make reporting cleaner, we also include a formatted summary : per-group metrics table, gaps table, ECE table, and a short “quick read” printout.

	n	BaseRate_P(Y=1 A)	Independence_P(Y=1 A)	Separation_TPR	Separation_FPR	Sufficiency_PPV	Sufficiency_NPV
Female	3627.0	0.2473	0.1478	0.4716	0.0414	0.7892	0.8467
Male	4373.0	0.1628	0.0922	0.4213	0.0281	0.7444	0.8962
Independence gap (PPR) Separation gap (TPR) Separation gap (FPR) Sufficiency gap (PPV) Sufficiency gap (NPV) Disparate Impact (min/max PPR)							
0	0.0556	0.0502	0.0133	0.0448	0.0496	0.6236	
ECE (10 bins)							
Group							
Female	0.0129						
Male	0.0185						

4 Conclusion and remark

Using feature engineering and robust hyperparameter selection with nested stratified CV, XG-Boost is selected as the final model. The fairness assessment with respect to Gender, computed on OOF predictions, highlights group differences across Independence and smaller—but non-zero—differences across Separation/Sufficiency, with calibration checked via ECE and calibration curves.

To go a bit outside of the program, we could use a "well-known" algorithm that comes from DeepLearning : TabPFN v2.5 which is extremely powerful on tabular datas.